Relatório: Sincronização do Metrô (sync_car)

Aluno: João Lucas de Oliveira Gonçalves (jlogb)
Repositório: https://github.com/LucasBaima/infra-sync-car
Sistema operacional: Linux Mint (desenvolvimento no Windows, compilação e testes no Linux Mint)

1. Objetivo

O enunciado pede a sincronização, em C com pthreads, do embarque de passageiros em vagões de metrô: o passageiro espera um vagão com lugar livre, e o vagão só parte quando está cheio ou quando não há mais ninguém esperando, sempre depois que todos os passageiros liberados sentarem. Entrego sync_car.h e sync_car.c com as quatro funções pedidas, o Makefile e o README, validados pelo sync_car-runner.c do professor.

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
station_init inicializa a estação	OK	Chamada pelo runner no início; contadores zerados
load_car retorna na hora sem passageiros esperando	OK	Teste inicial do runner (alarme de 1 s)
load_car retorna na hora com 0 lugares	OK	Runner com 100 passageiros esperando (alarme de 2 s)
Não embarcar mais passageiros que lugares	OK	Em todo vagão, "new passenger(s)" = "expected" (Figura 2)
Vagão só parte depois de todos os on_board	OK	Runner nunca acusou "returned early"
Vagão parte sem lotar quando não há mais quem esperar	OK	Último vagão: 49 lugares, partiu com os 12 restantes (Figura 2)
Vários passageiros liberados antes de qualquer on_board	OK	wait_for_car reserva o lugar e solta o mutex sem esperar sentar
Somente pthread_mutex_* e pthread_cond_*, um único mutex	OK	sync_car.h: 1 mutex e 2 variáveis de condição
Sem busy wait	OK	Toda espera usa pthread_cond_wait
Makefile compila e limpa	OK	make gera sync_car; make clean remove executável e .o
Correto em execuções repetidas	OK	200 de 200 execuções no laço de estresse (evidencias.log)

3. Como Reproduzir

Compilar (dentro da pasta jlogb):

```
make clean
make
```

Executar (saída esperada ao final: Looks good!):

```
./sync_car
```

Teste de estresse (200 execuções seguidas):

```
for i in $(seq 1 200); do ./sync_car > /dev/null || { echo "FALHOU na rodada $i";
break; }; done; echo "fim"
```

4. Arquitetura (resumida)

Arquivos e responsabilidades:

- **sync_car.h** → struct station (1 mutex, 2 variáveis de condição, 3 contadores) e protótipos.
- **sync_car.c** → station_init, station_load_car, station_wait_for_car e station_on_board.

- **sync_car-runner.c** → teste do professor, sem alterações.
- **Makefile** → compila com gcc -Wall -Wextra -pthread; alvo clean.

Decisões técnicas:

- **Três contadores: waiting_passengers, free_seats, boarding_passengers** → separar quem espera, quantos lugares restam e quem foi liberado mas não sentou → a condição de partida vira uma única expressão: boarding == 0 e (free_seats == 0 ou waiting == 0).
- **Duas variáveis de condição sobre um único mutex** → passageiros e vagão esperam por eventos diferentes → o vagão usa broadcast em car_arrived; o último passageiro usa signal em all_boarded.
- **pthread_cond_wait dentro de while, não de if** → um passageiro pode acordar e encontrar o lugar já tomado (ou acordar sem motivo) → a condição é sempre reavaliada antes de reservar.
- **on_board só sinaliza quando boarding_passengers chega a 0** → o vagão só pode decidir partir nesse momento → evita acordá-lo à toa.
- **free_seats = 0 quando o vagão parte** → lugares não usados não podem ficar disponíveis → ninguém embarca num vagão que já saiu.
- **struct station no .h (o PDF indica o .c)** → o runner declara struct station station; no main e precisa conhecer o conteúdo da struct → sem isso o runner não compila.

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias

Estratégia	Nome curto	Contexto	Motivo da troca
S1	Contadores + mutex + 2 condições	Estado da plataforma em 3 contadores protegidos por um mutex; passageiros dormem em car_arrived e o vagão em all_boarded. Implementada em etapas: esqueleto, wait_for_car, on_board, load_car.	Não houve troca

5.2 Diário de Tentativas

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Esqueleto (struct, station_init, Makefile) e make no Windows	Parcial: sync_car.c compila; executável não gerado	Runner usa APIs POSIX (alarm, SIGALRM, random) ausentes no Windows; decidi testar no Linux Mint	04/11	Figura 1
2	S1	Implementar station_wait_for_car	OK: compila sem warnings		04/11	Histórico do git
3	S1	Revisar o diff antes do commit seguinte	Falha: mudança não commitada apagava o wait_for_car	Arquivo reescrito a partir de cópia antiga (havia pastas jlog e jlogb); descartei a mudança e unifiquei tudo em jlogb	04/11	Histórico do git
4	S1	Implementar station_on_board	OK: compila sem warnings		04/11	Histórico do git
5	S1	Implementar station_load_car	OK: compila sem warnings		06/11	Histórico do git
6	S1	Primeira execução no Linux Mint	OK: Looks good!; único warning é do runner		07/11	Figura 2, evidencias.log
7	S1	Teste de estresse com 200 execuções	OK: 200 de 200		07/11	evidencias.log

As tentativas 1 a 5 foram feitas no Windows, onde o runner não roda; a prova delas é o histórico de commits no GitHub. O evidencias.log cobre a sessão no Linux Mint (tentativas 6 e 7).

6. Evidências

6.1 Log automático

evidencias.log (dentro do .tar), gerado com script -a evidencias.log na sessão do Linux Mint: date, whoami e pwd, seguidos de make clean, make, ./sync_car e o laço de 200 execuções.

6.2 Prints

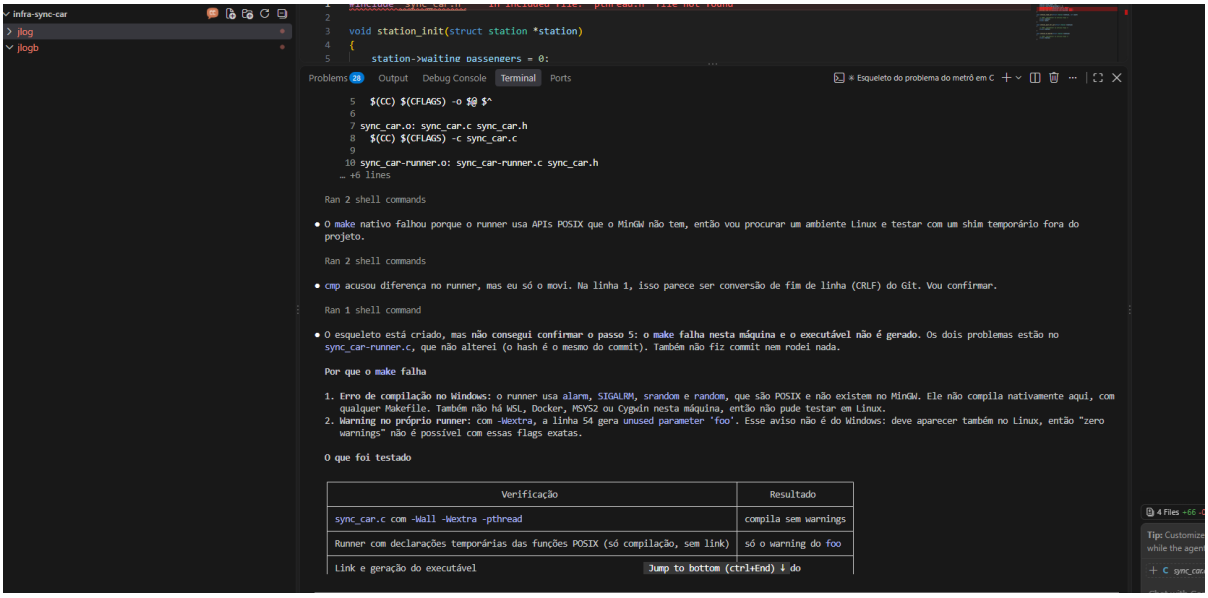


Figura 1: erro principal. make no Windows falha porque o runner usa APIs POSIX; o sync_car.c compila sem warnings.

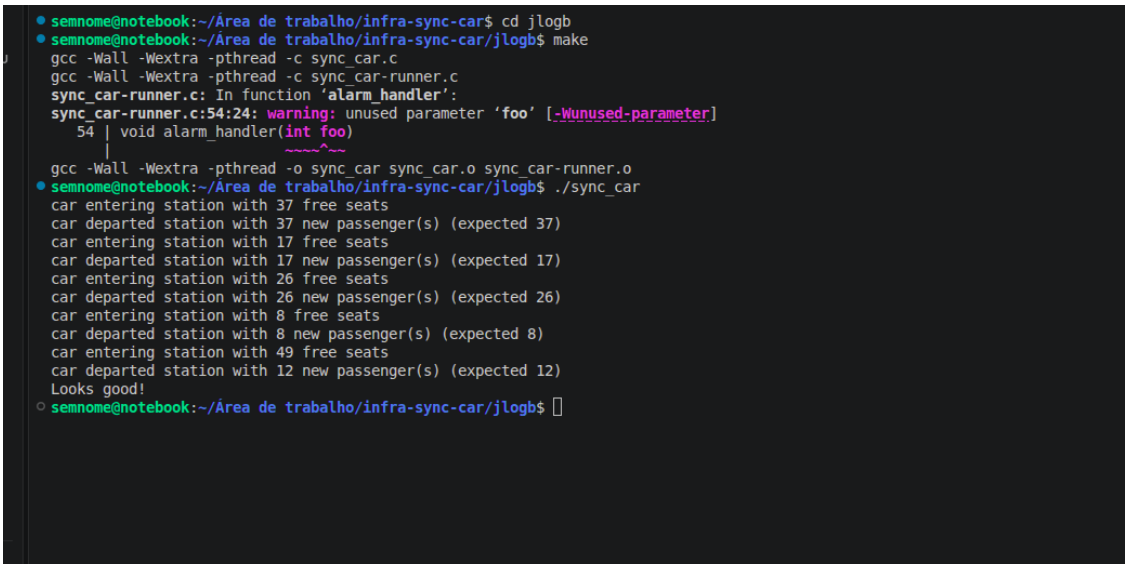


Figura 2: validação final no Linux Mint. Compilação, 100 passageiros embarcados e Looks good!

7. Uso de IA

Onde usei IA: Usei IA (Claude) como assistência para compreender a sincronização (condição de corrida, mutex e variáveis de condição aplicados ao embarque nos vagões) e para implementar algumas linhas do código. O código não foi feito inteiramente por IA: cada trecho foi lido, entendido e revisado por mim antes do commit.

Prompts principais:

- Explicar o problema do metrô em linguagem simples e o papel do sync_car-runner.c.
- Quais informações a estação precisa guardar para decidir embarque e partida.
- Implementar uma função por etapa, conferindo o arquivo atual antes de editar.

- Por que while e não if no pthread_cond_wait; quando usar signal ou broadcast.

O que validei manualmente e como: compilei com -Wall -Wextra (zero warnings no meu código); rodei o runner no Linux Mint (Looks good!) e 200 vezes seguidas; conferi a condição de partida contra os casos do runner (0 lugares e plataforma vazia); revisei o diff antes de cada commit, o que me fez detectar a sobrescrita do sync_car.c (tentativa 3).

8. Reflexão Final

A decisão central foi modelar a plataforma em três contadores: com eles, cada espera vira uma condição simples e a regra de partida cabe numa única expressão. O principal aprendizado é que, em concorrência, passar no teste uma vez não prova que o código está certo, porque o resultado depende do escalonamento; por isso rodei o teste 200 vezes. Na próxima, evitaria duas coisas: manter duas pastas de entrega (a causa da sobrescrita do arquivo) e deixar o teste em Linux só para o fim. Com mais tempo, testaria no Linux desde o primeiro esqueleto.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim
Rodei os testes e evidencias.log foi gerado	Sim
Tenho prints obrigatórios	Sim (Figuras 1 e 2)
Testei pelo menos um caso limite e um caso inválido	Parcial: limites sim (0 lugares, plataforma vazia); entrada inválida não tratada (seção 10)
Preenchi a seção de uso de IA	Sim
Revisei o relatório e removi frases genéricas / vazias	Sim

10. Se eu tivesse mais 2 horas

- Tratar entrada inválida: count negativo hoje seria aceito como número de lugares.
- Rodar com ThreadSanitizer (gcc -fsanitize=thread) para detectar acesso a dado compartilhado sem trava.
- Verificar o retorno das chamadas pthread_* em vez de assumir sucesso.
- Criar station_destroy (pthread_mutex_destroy e pthread_cond_destroy) para liberar os recursos.
- Adicionar um alvo make test com o laço de estresse e rodá-lo também com a máquina sob carga.